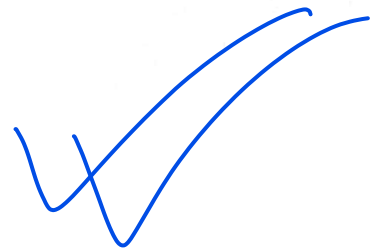


Answer to: question: 0101(a)

<u>Input symbol</u>	<u>Stack</u>	<u>Output Expression</u>
a	null	a
↑	↑	a
2	↑	a2
-	• -	a2↑
(	-(	a2↑
b	-(	a2↑b
+	-(+	a2↑b
c	-(+	a2↑bc
/	-(+/	a2↑bc
d	-(+/	a2↑bcd
*	-(+*	a2↑bcd/
a	-(+*	a2↑bcd/a
+	-(+	a2↑bcd/a*+
b	-(+	a2↑bcd/a*+b
+	-(+	a2↑bcd/a*+b+
-	-(	a2↑bcd/a*+b+-
)	null	



(2)

01(b)

Given postfix : 2 3 - 2 1 * +

<u>input</u>	<u>stack state</u>	output
2	2,	
3	2, 3	
-	-1	
2	-1, 2	
1	-1, 2, 1	
*	-1, 2	
+	-1	



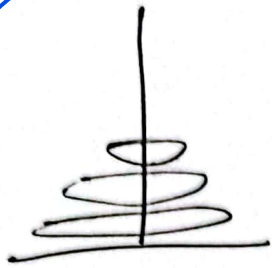
Answer = -1

01(c)

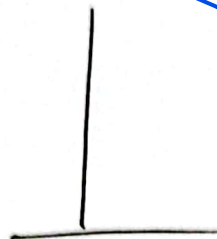
```

void
hanoi(n, start, end) {
    if (n == 1)
        printf("%d → %d", start, end);
    else {
        thirdPeg = 6 - (start + end);
        hanoi(n-1, start, thirdPeg);
        printf("%d → %d", start, end);
        hanoi(n-1, thirdPeg, end);
    }
}

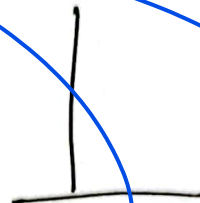
```



start = 1



third Peg = 2



End = 3

Sequence for $n = 3$:

1 → 3

1 → 2

3 → 2

1 → 3

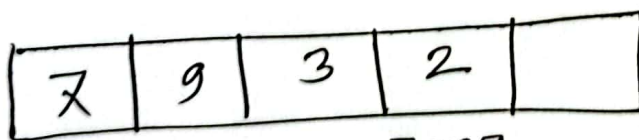
2 → 1

2 → 3

1 → 3

total moves = 7

01 (d)



front

rear

queue $\Rightarrow$ 9

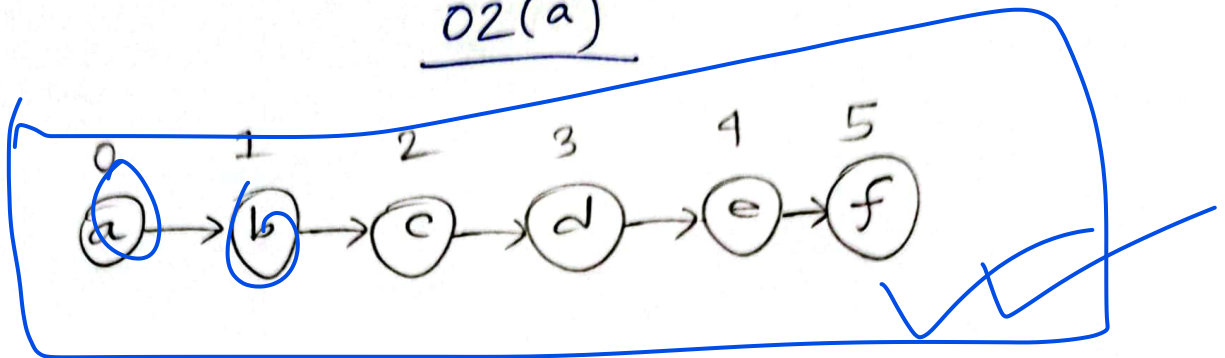
```

void enqueue (int n) {
    if (rear >= q.size() - 1) {
        cout << "Queue is OVERFLOW";
    } ..... // Rest code
}

```

}

(4)

Answer to question: 0202(a)02(b)

Adjacency Matrix:

	0	1	2	3	4	5
0	0	1	0	0	0	0
1	0	0	1	0	0	0
2	0	0	0	1	0	0
3	0	0	0	0	1	0
4	0	0	0	0	0	1
5	0	0	0	0	0	0

Space required: $O(|V|^2)$

Adjacency list:

0 → 1 → NULL

1 → 2

2 → 3

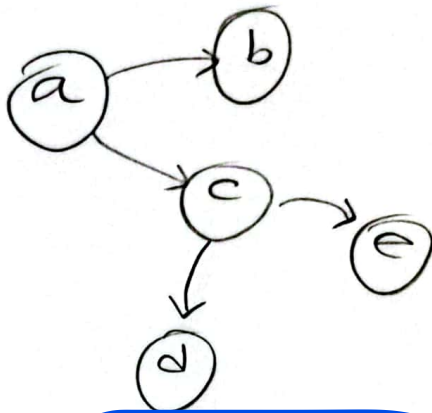
3 → 4

4 → 5

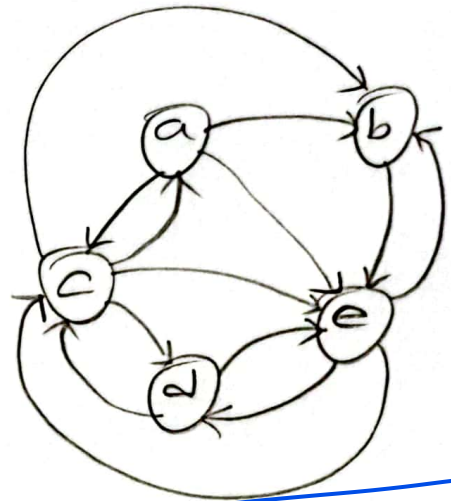
5

Space: $O(|V| + |E|)$
or, $O(V + E)$

02(c)

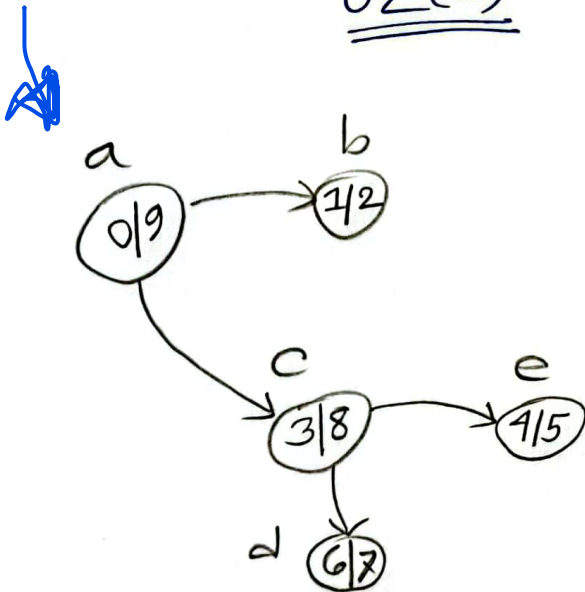


Sparse graph



Dense graph

02(d)



dfs
sequence:

~~a~~ → b → c → e → d

(b) (e) (d) (c) (a)

02(e)

Topological Sort Algorithm:

1. while (dfs is running):

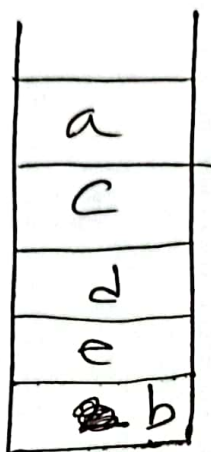
push the black color node inside a stack;

2. dfs ends;

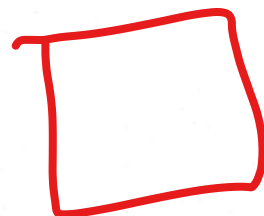
3. print all elements of the stack from top to bottom.

6

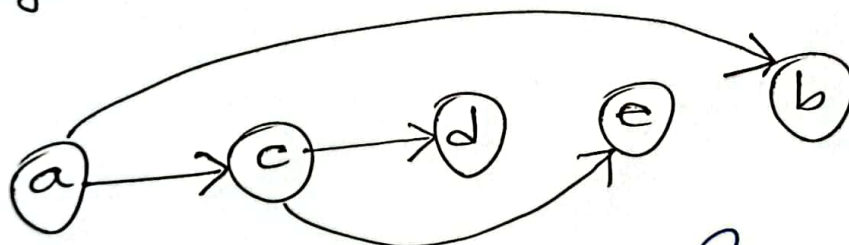
Using the dfs process in ~~step~~ 2(d):
putting in the stack according
to finishing time:



Stack

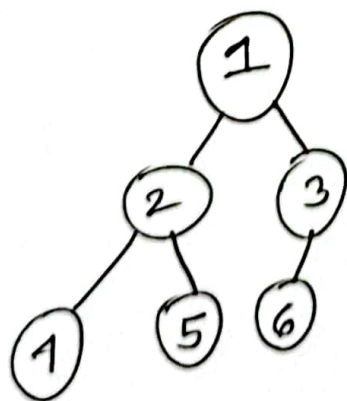


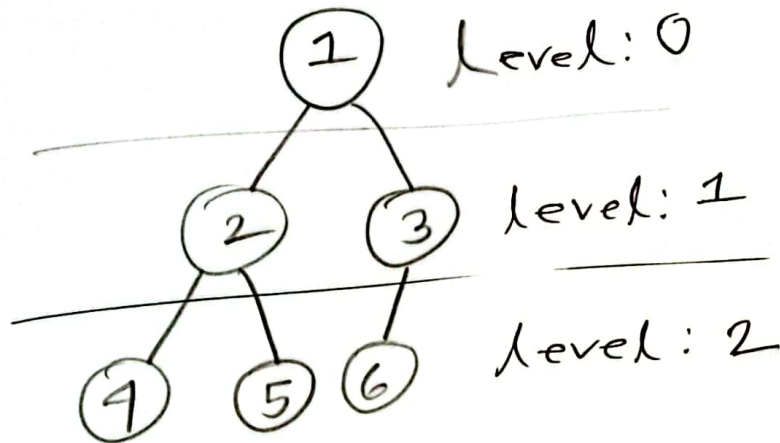
Topological sequence: ~~{~~ a, c, d, e, b



Question - 03

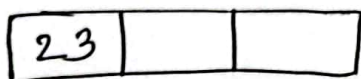
03(a)



03(b)pre-order: 1, 2, 4, 5, 3, 6in-order: 4, 2, 5, 1, 6, 3post-order: 4, 5, 2, 6, 3, 1level-order: 1, 2, 3, 4, 5, 6height: the tree's height = $2 - 0$
2 2

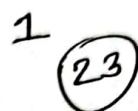
03(c)

Enqueue (23) :



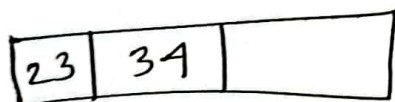
front = 0
rear = 0

queue

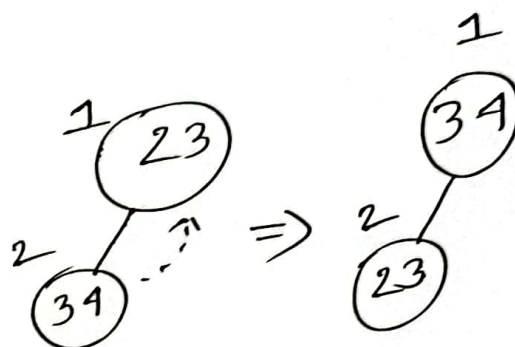


priority Queue

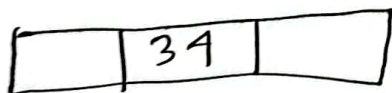
Enqueue (34) :



front = 0 rear = 1



Dequeue () :



front = rear = 1

print (23)

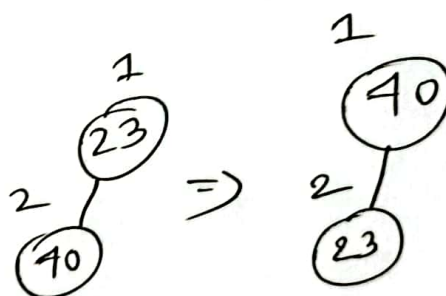
Enqueue (40) :



f = 1 r = 2

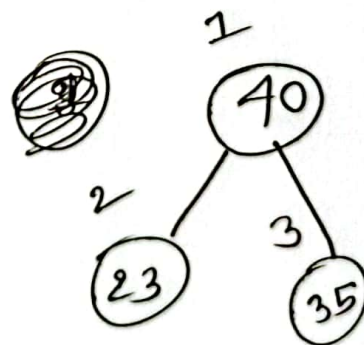


print (34)

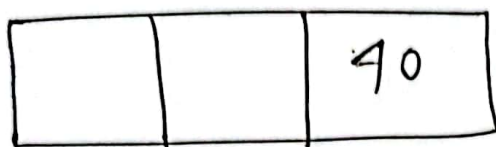


Enqueue (35) :

print ("queue overflow")

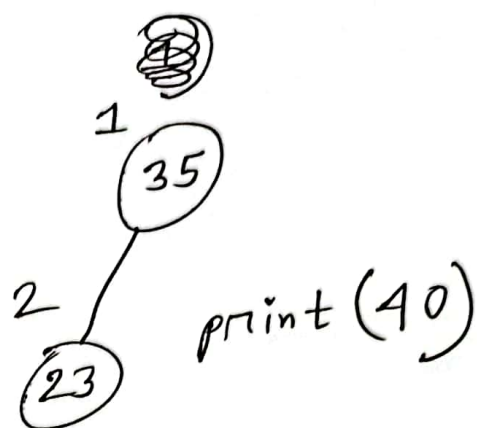


Dequeue() :



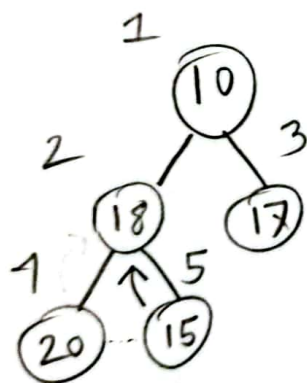
front = rear = 2

print(34)

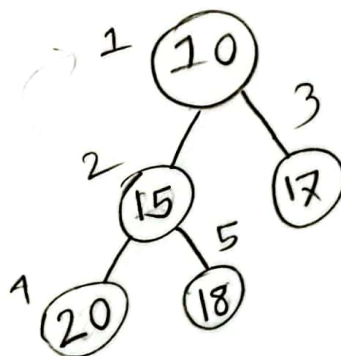


Q3 (d)

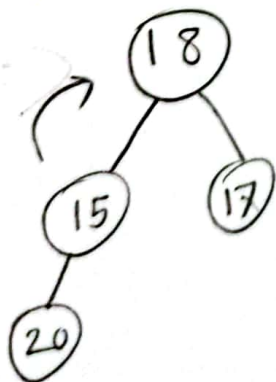
Building Min-Heap:



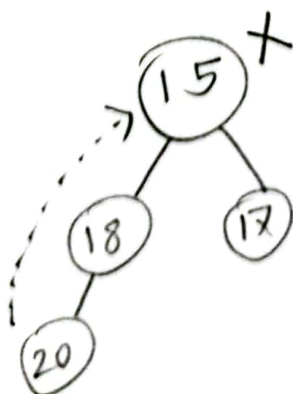
⇒



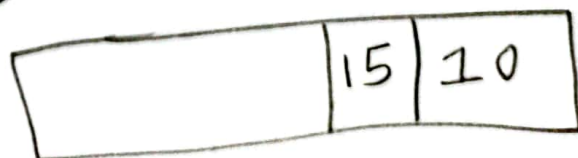
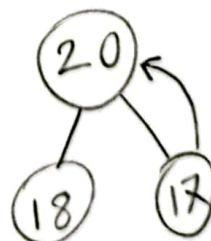
Heap sort:

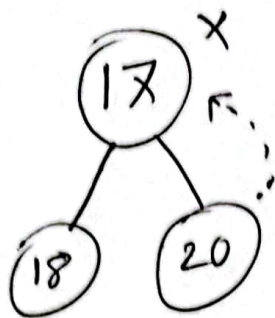


⇒

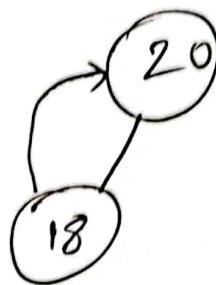


⇒





$\Rightarrow$



$\Rightarrow$

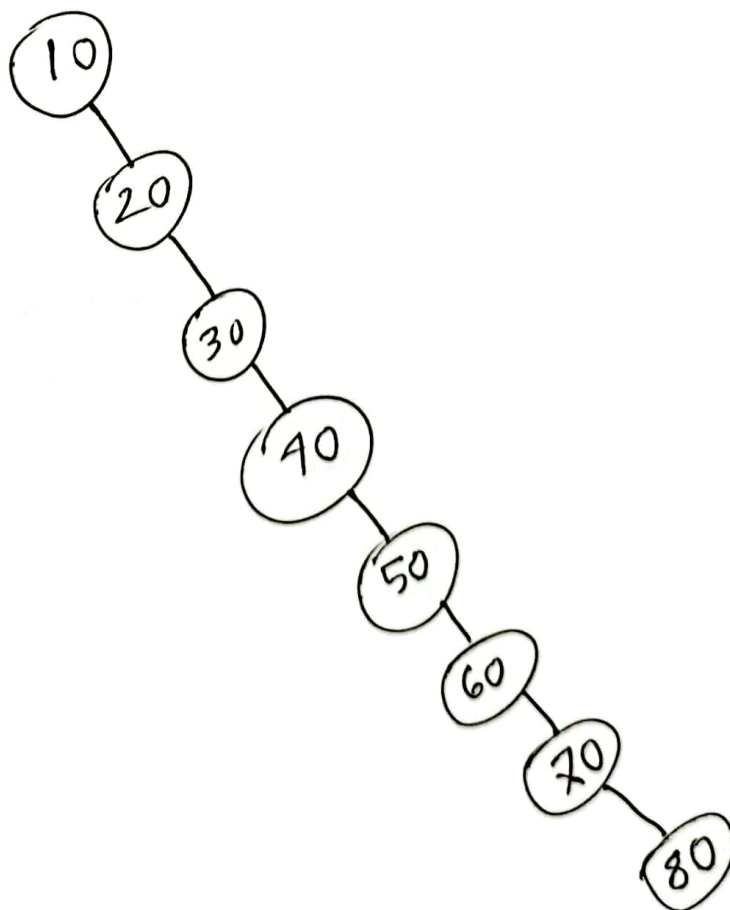


		17	15	10
--	--	----	----	----

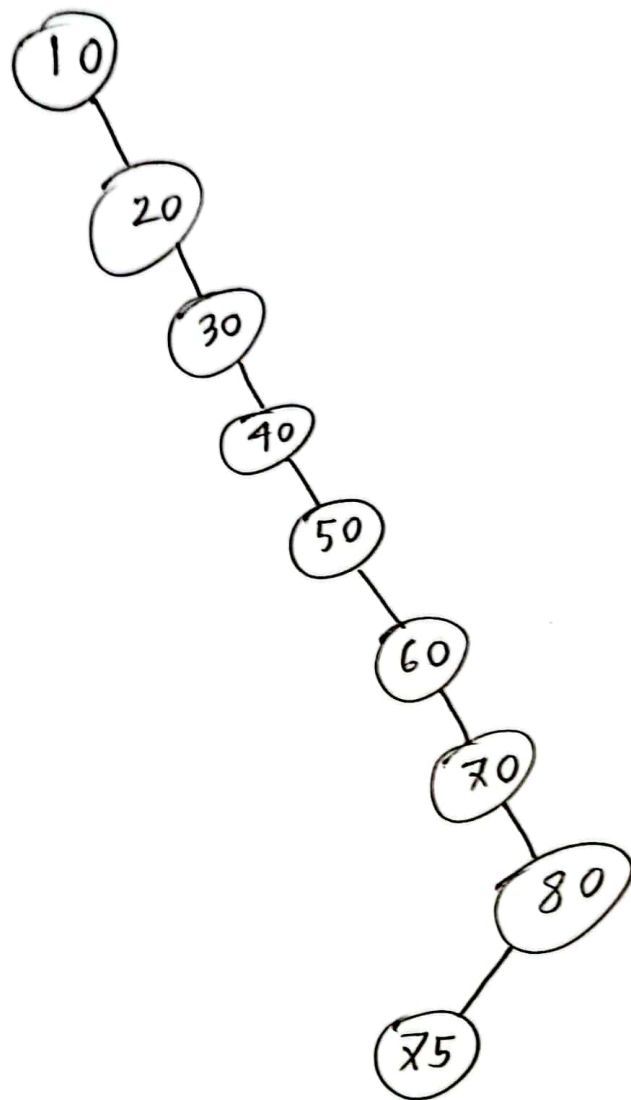
20	18	17	15	10
----	----	----	----	----

04 - Question - 04

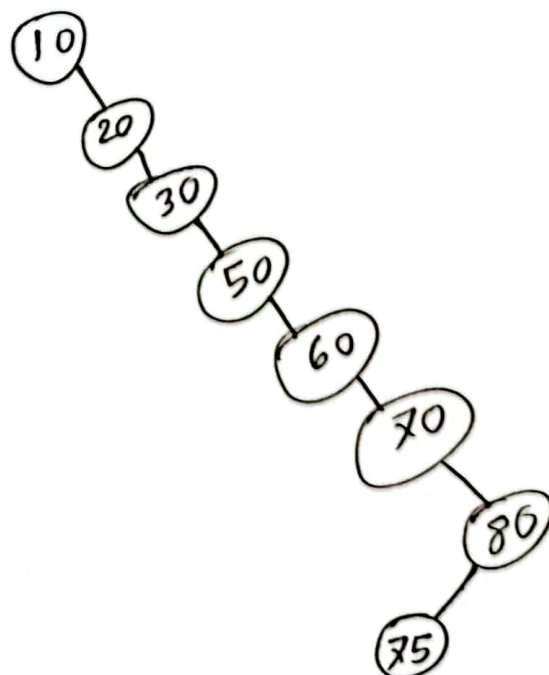
04 (a)



insert (75):



Delete (40):



Re: 6

Ex:

04(b)

✓

Array:

[0]	[1]	[2]	[3]	[4]	[5]	[6]
0	1	2	3	4	5	6

Linked List:

04(c)

$$S1 = \{10, 20, 30, 40\} \Rightarrow S1^* = 40$$

$$S2 = \{50, 60, 70\} \Rightarrow S2^* = 70$$

$$\text{Find}(10) \neq \text{Find}(50)$$

$$\Rightarrow S1^* \neq S2^*$$

$$40 \neq 70 ; \text{which is true;}$$

$\Rightarrow$ So, 10 & 50 are not in same set

04 (d)

i) Weighted Undirected Multigraph:

two ~~different~~ towns may be ~~connected with different roads~~ bidirectional ~~or~~ two-way roads. These roads vary in distance which can be denoted by a weighted edge.

ii) QUEUE:

If a doctor wants to show justice to his patients, he will have to ~~serve~~ serve those patients who came to him first.

iii) STACK:

~~putting~~ ~~every~~ Pushing every character on a stack as inputs are being taken, and printing the stack, afterwards from top to bottom is the best way to reverse a string.